

Biến đổi và bất biến trong bài toán

Chen Xue

Nguồn gốc: Change and invariance in problems

IOI China candidate team papers / 2007 / day 2 / Chen Xue

Tóm tắt

Trong thi tin học, rất nhiều đề bài về bản chất là quá trình tìm hoặc duy trì các biến. Nhưng đôi khi biến có rất nhiều kiểu thay đổi; nếu chỉ duy trì trực tiếp các biến thì độ phức tạp thời gian và bộ nhớ thường trở nên thấp hiệu quả. Nếu có thể tìm ra những đại lượng “không đổi” ẩn trong sự thay đổi của biến, bài toán thường được giải quyết rất gọn. Bài viết này giới thiệu ngắn gọn một vài kỹ thuật tìm phần “bất biến” trong các biến, qua đó làm đơn giản hóa bài toán.

Từ khóa. Biến đổi; biến; bất biến; hằng lượng; duy trì.

1 Dẫn nhập

Biến là khái niệm thường gặp nhất trong tin học. Những dạng cơ bản nhất gồm biến vòng lặp, biến tích lũy và biến quyết định. Duy trì các biến ấy cũng là thao tác căn bản trong tin học. Quanh việc duy trì biến, tin học có nhiều cấu trúc dữ liệu như cây nhị phân cân bằng, ngăn xếp, hàng đợi, v.v.

Tuy nhiên, một số bài toán có quy mô rất lớn; chỉ riêng tổng số thao tác cần thiết để duy trì các biến đã có thể tạo ra độ phức tạp thời gian hoặc bộ nhớ không thể chấp nhận. Vì vậy ta cần rút gọn các biến, tìm ra thông tin hữu ích trong chúng, từ đó giảm số thao tác duy trì, thậm chí biến một số đại lượng thành “hằng lượng”.

Sau đây ta sẽ xem cụ thể các kỹ thuật kiểu này được dùng trong tin học như thế nào.

2 Từ tập biến đến hằng lượng

Gom các biến hữu ích lại thành một tập hợp rồi lợi dụng tính chất đặc biệt của tập ấy, chẳng hạn thông qua phép cộng hoặc phép nhân, là một trong những cách thường gặp nhất để biến biến lượng thành hằng lượng. Ta bắt đầu bằng một bài toán như vậy.

2.1 Ví dụ 1: Ants

Một số con kiến bò trên đoạn thẳng dài l cm với vận tốc 1 cm/s; khi bò đến đầu mút của đoạn thẳng thì chúng rơi xuống. Khi hai con kiến gặp nhau, cả hai lập tức quay đầu. Biết l và vị trí ban đầu của từng con kiến, nhưng không biết hướng ban đầu của chúng. Hãy tìm thời điểm sớm nhất mà tất cả kiến đều rơi xuống và thời điểm muộn nhất mà tất cả kiến đều rơi xuống.

Số kiến tối đa là 1 000 000.

Phân tích

Ban đầu ta chỉ biết vị trí của mỗi con kiến, không biết hướng đi. Nếu liệt kê trực tiếp mọi cách chọn hướng, riêng số phương án đã là $2^{1000000}$, hoàn toàn không thể xử lý. Vì vậy không thể liệt kê hướng của từng con kiến; ta nên bắt đầu từ một bài toán đơn giản hơn.

Giả sử đã biết hướng ban đầu của mọi con kiến. Khi ấy nhiệm vụ còn lại là mô phỏng toàn bộ quá trình gặp nhau và rơi khỏi đoạn thẳng, rồi tìm thời điểm con kiến cuối cùng rơi xuống. Trong quá trình này, biến phức tạp nhất chính là hướng của từng con kiến. Rõ ràng trong trường hợp xấu nhất, số lần gặp nhau có thể đạt cấp N^2 , nên chỉ việc duy trì hướng và vị trí của từng con kiến đã không đáp ứng được yêu cầu thời gian.

Ta xét chi tiết thao tác cơ bản nhất: hai con kiến đụng đầu nhau. Giả sử kiến A bò từ trái sang phải, gặp kiến B bò từ phải sang trái tại điểm X rồi quay lại; đồng thời kiến B cũng quay lại và bò sang phải từ X .

Ghi thông tin hữu ích của hai con kiến, tức vận tốc V_i (một vectơ) và vị trí hiện tại W_i , vào cùng một tập:

$$U = \{(V_a, W_a), (V_b, W_b)\}.$$

Ngay trước khi A và B gặp nhau,

$$U = \{(1, X), (-1, X)\}.$$

Ngay sau khi hai con kiến gặp nhau,

$$U' = \{(-1, X), (1, X)\}.$$

Ta có $U = U'$. Nói cách khác, đối với từng con kiến thì vận tốc đã thay đổi, nhưng nếu nhìn hai con kiến như một tập hợp thì tập ấy không đổi. Do đó, mọi lần gặp nhau giữa các con kiến trong cùng một tập đều không làm thay đổi tập.

Xem tất cả kiến như một tập lớn

$$V = \{(V_1, W_1), (V_2, W_2), \dots, (V_n, W_n)\}.$$

Theo nhận xét trên, mọi lần gặp nhau bên trong V đều không ảnh hưởng đến tập V . Đồng thời, từ tập V ta biết rằng tuy không thể xác định thời điểm rơi T_i của từng con kiến riêng lẻ, ta có thể xác định được tập các thời điểm rơi:

$$T = \{\text{thời gian để con kiến thứ } i \text{ đi theo hướng ban đầu đến một đầu mút} \mid 1 \leq i \leq N\}.$$

Quay lại bài toán gốc. Với thời điểm sớm nhất để tất cả rơi xuống, ta chỉ cần cho mỗi con kiến bò về đầu mút gần nó nhất; khi ấy giá trị lớn nhất trong T là nhỏ nhất. Tương tự, với thời điểm muộn nhất, cho mỗi con kiến bò về đầu mút xa nó nhất; khi ấy giá trị lớn nhất trong T là lớn nhất.

Ta thu được thuật toán $O(N)$, cũng là cận dưới về mặt lý thuyết vì phải đọc toàn bộ vị trí đầu vào.

Nhận xét

Nhờ khái niệm tập hợp, các biến ban đầu rất lớn như hướng và vị trí của từng con kiến được chuyển thành những “hằng lượng”. Từ những hằng lượng ấy, không cần duy trì quá trình cụ thể vẫn có thể trực tiếp suy ra kết quả cuối cùng.

Phép nhìn theo tập hợp có ứng dụng rất rộng trong các bài toán biến đổi biến thành hằng lượng. Ta thường có thể dùng điều kiện đề bài để xây dựng tập hợp, rồi dựa vào tính chất đặc biệt giữa các phần tử trong tập để chứng minh vô nghiệm hoặc tối ưu hóa thuật toán.

Ví dụ này cũng cho thấy: quan sát kỹ, bắt đầu từ chi tiết nhỏ và tìm mối liên hệ giữa các biến là con đường để phát hiện cách biến biến lượng thành “hằng lượng”.

3 Giữ bất biến bằng cách đổi biến

3.1 Ví dụ 2: Navigation Game

Cho một bảng N hàng và M cột. Hàng N tượng trưng cho bờ xuất phát, hàng 1 tượng trưng cho bờ bên kia; $N - 2$ hàng ở giữa là biển. Mục tiêu là điều khiển một con thuyền, xuất phát từ một bến bất kỳ ở bờ này, ký hiệu bằng H , đi đến một bến bất kỳ ở bờ bên kia trong thời gian ngắn nhất.

Thuyền chỉ có thể đi sang trái, sang phải hoặc đi lên, mỗi lần một ô. Trừ khi cập bờ, tức phải đến một bến và kết thúc trò chơi, thuyền không được đi vào đất liền.

Một khi đã rời khỏi một ô, thuyền không bao giờ được quay lại ô đó. Đi lên luôn chỉ tốn một đơn vị thời gian. Nhưng đi ngang trái hoặc phải nhiều lần liên tiếp sẽ ngày càng tốn kém: nếu trước một lần đi ngang, thuyền đã đi ngang liên tiếp x lần, thì lần đi ngang này tốn $x + 1$ đơn vị thời gian.

Trên biển có thể gặp các ô sau:

- 0: chướng ngại vật, không thể đi vào.
- F: vòng quay số mệnh. Thuyền phải đi qua số lẻ ô loại này trong toàn hành trình thì mới an toàn đến bờ bên kia.
- B: đá chước phúc. Đi vào ô này không tốn thời gian.
- S: bùa giông bão. Đi vào ô này tốn gấp đôi thời gian bình thường.

Dữ liệu thỏa $N, M \leq 1000$.

Phân tích

Theo yêu cầu đề bài, thuyền không được quay lại, đồng thời chỉ có thể đi lên, sang trái hoặc sang phải. Vì vậy ta xét quy hoạch động theo từng hàng. Đặt $f_{t,i,j}$ là chi phí nhỏ nhất để đi từ điểm xuất phát đến ô (i, j) , trong đó số ô F đã đi qua có tính chẵn lẻ là t . Đặt $C_{i,j}$ là chi phí để đi từ $(i + 1, j)$ lên (i, j) .

Ta có chuyển tiếp

$$f_{t,i,j} = \min_k \{f_{t',i+1,k} + C_{i,k} + \text{cost}_{i,k,j}\},$$

trong đó $\text{cost}_{i,k,j}$ là chi phí đi ngang trong hàng i từ (i, k) đến (i, j) theo quy tắc của đề, và giữa hai ô ấy không có chướng ngại vật. Giá trị t' được xác định từ t bằng cách lấy xor với số ô F trên đoạn ngang đang đi.

Nếu tạm bỏ qua các ràng buộc 0, B, S, thì

$$\text{cost}_{i,k,j} = \frac{|j - k|(|j - k| + 1)}{2}.$$

Trước hết xét hướng từ trái sang phải, tức $j > k$. Khi đó

$$f_{t,i,j} = \min_k \left\{ f_{t',i+1,k} + C_{i,k} + \frac{(j - k)(j - k + 1)}{2} \right\}.$$

Với một hàng và một trạng thái chắn lẻ cố định, đặt

$$U_j = f_{t,i,j}, \quad V_k = f_{t',i+1,k} + C_{i,k}.$$

Khi ấy

$$U_j = \min_k \left\{ V_k + \frac{(j-k)(j-k+1)}{2} \right\}.$$

Giả sử k là quyết định tối ưu cho j , và so sánh nó với một quyết định l ($j > k > l$). Ta có

$$V_k + \frac{(j-k)(j-k+1)}{2} < V_l + \frac{(j-l)(j-l+1)}{2},$$

tương đương

$$\frac{(2V_k + k(k-1)) - (2V_l + l(l-1))}{2(k-l)} < j.$$

Đặt

$$X_i = 2V_i + i(i-1), \quad Y_i = 2i.$$

Khi độ dốc giữa hai điểm (X_k, Y_k) và (X_l, Y_l) nhỏ hơn j , quyết định k tốt hơn l . Đây là mô hình quy hoạch động tối ưu hóa bằng độ dốc quen thuộc. Theo cách làm chuẩn, ta duy trì một hàng đợi biểu diễn phần bao lồi dưới của các điểm (X_i, Y_i) , nhờ đó tìm được giá trị tối ưu của U_j trong thời gian khấu hao $O(1)$ cho mỗi j .

Cụ thể, dùng một mảng stack làm hàng đợi của phần bao lồi dưới, với hai con trỏ đầu và cuối. Các chỉ số trong hàng đợi tăng dần:

$$\text{stack}_1 < \text{stack}_2 < \dots < \text{stack}_{en},$$

và độ dốc giữa hai điểm kề nhau cũng tăng dần:

$$\text{slope}(\text{stack}_1, \text{stack}_2) < \text{slope}(\text{stack}_2, \text{stack}_3) < \dots < \text{slope}(\text{stack}_{en-1}, \text{stack}_{en}).$$

Duyệt j từ nhỏ đến lớn. Nếu độ dốc giữa hai phần tử đầu hàng đợi nhỏ hơn j , ta liên tục bỏ phần tử đầu. Khi dừng, phần tử đầu chính là quyết định tối ưu cho j . Sau đó thêm j vào cuối hàng đợi và duy trì tính chất bao lồi. Hướng từ phải sang trái được xử lý tương tự.

Điểm đặc biệt của bài này là sự xuất hiện của B và S. Chi phí thay đổi khi đi qua một ô B hoặc S phụ thuộc vào điểm bắt đầu, nên không thể bê nguyên cách trên vào bài toán.

Xét lại công thức gốc

$$f_{t,i,j} = \min_k \{ f_{t',i+1,k} + C_{i,k} + \text{cost}_{i,k,j} \}.$$

Vẫn xét từ trái sang phải, $j > k$. Dù giữa (i, k) và (i, j) có các ô B hoặc S, đại lượng $\text{cost}_{i,k,j}$ luôn có thể viết thành một hàm bậc hai theo $j - k$:

$$\text{cost}_{i,k,j} = \frac{(j-k+1)(j-k)}{2} + a(j-k) + b,$$

trong đó a và b là các biến cần duy trì.

Giả sử k cố định và

$$\text{cost}_{i,k,j} = \frac{(j-k+1)(j-k)}{2} + a(j-k) + b.$$

Khi mở rộng sang ô $j+1$, có ba trường hợp:

1. Nếu $(i, j+1)$ không phải B cũng không phải S, thì

$$\text{cost}_{i,k,j+1} = \frac{(j-k+2)(j-k+1)}{2} + a(j-k+1) + b - a.$$

2. Nếu $(i, j + 1)$ là B, đi vào ô này không tốn thời gian, nên

$$\text{cost}_{i,k,j+1} = \frac{(j - k + 2)(j - k + 1)}{2} + (a - 1)(j - k + 1) + b - a.$$

3. Nếu $(i, j + 1)$ là S, thời gian bị nhân đôi, nên

$$\text{cost}_{i,k,j+1} = \frac{(j - k + 2)(j - k + 1)}{2} + (a + 1)(j - k + 1) + b - a.$$

Vì vậy chỉ cần cập nhật kịp thời a, b là có thể thu được công thức mới của $\text{cost}_{i,k,j+1}$. Đồng thời ta thấy ý nghĩa của a : trên đoạn $(i, k + 1)$ đến (i, j) ,

$$a = \sum \mathbf{1}_S - \sum \mathbf{1}_B.$$

Trường hợp khó hơn là j cố định còn k thay đổi. Quét lại từ đầu chắc chắn làm thuật toán kém hiệu quả. Hơn nữa, số biến $\text{cost}_{i,k,j}$ có cấp $O(n^3)$; ngay cả khi tính được từng biến trong thời gian khấu hao $O(1)$, tổng thời gian vẫn là $O(n^3)$. Vì vậy ta cần tính nhanh chỉ những thông tin thật sự hữu ích.

Giả sử hiện biết

$$\text{cost}_{i,k,j} = \frac{(j - k + 1)(j - k)}{2} + a(j - k) + b,$$

và cũng biết

$$\text{cost}_{i,k,l} = \frac{(l - k + 1)(l - k)}{2} + a'(l - k) + b', \quad l > k.$$

Ta muốn tính $\text{cost}_{i,l,j}$. Thay vì tìm cách tính lại trực tiếp, ta điều chỉnh biến khác để giữ a và b bất biến. Chi phí từ (i, k) đến (i, j) gồm hai phần: từ (i, k) đến (i, l) và từ $(i, l + 1)$ đến (i, j) . Phần đầu chính là $\text{cost}_{i,k,l}$; phần sau chứa chi phí ta cần, cộng với lượng dư do xuất phát từ k thay vì từ l .

Từ đó có

$$\begin{aligned} f_{t',i+1,l} + C_{i,l} + \text{cost}_{i,l,j} \\ = f_{t',i+1,l} + C_{i,l} + \frac{(j - l + 1)(j - l)}{2} + a(j - l) + b - b'. \end{aligned}$$

Do đó ta chỉ cần hấp thụ b' vào biến mới

$$g_{t',i,l} = f_{t',i+1,l} + C_{i,l} - b',$$

mà không phải thay đổi a và b . Cách làm này không ảnh hưởng đến các giá trị $\text{cost}_{i,l,j'}$ về sau, vì quá trình chuyển từ $\text{cost}_{i,l,j}$ sang $\text{cost}_{i,l,j'}$ chỉ phụ thuộc vào đoạn tiếp tục mở rộng.

Như vậy chuyển tiếp có thể viết lại thành

$$f_{t,i,j} = \min_k \{g_{t',i,k} + \text{cost}_{i,k,j}\} = \min_k \left\{ g_{t',i,k} + \frac{(j - k + 1)(j - k)}{2} + a(j - k) + b \right\}.$$

Với một giá trị $f_{t,i,j}$ cụ thể, a, b là cố định. Tương tự trường hợp không có B, S, so sánh hai quyết định k và l cho ta điều kiện dạng độ dốc:

$$\frac{(2g_{t',i,k} + k(k - 1)) - (2g_{t',i,l} + l(l - 1))}{2(k - l)} < j - \frac{a}{2}.$$

Đặt

$$X_k = 2g_{t',i,k} + k(k - 1), \quad Y_k = 2k.$$

Ta vẫn duy trì hàng đợi biểu diễn phần bao lồi dưới của các điểm (X_i, Y_i) . Cách thao tác giống trường hợp không có B, S, nên không lặp lại chi tiết.

Cần kiểm tra tính đúng đắn của việc dùng hàng đợi. Mỗi lần j tăng thêm 1, bất kể ô mới là B hay S, giá trị a chỉ thay đổi nhiều nhất 1. Vì vậy về phải trong điều kiện độ dốc vẫn biến thiên đơn điệu đủ để hàng đợi bao lồi hoạt động đúng.

Xét ngược từ phải sang trái cũng làm tương tự. Tổng độ phức tạp thời gian là

$$O(\text{số hàng} \cdot (\text{số cột} + \text{số thao tác hàng đợi})) = O(NM).$$

Nhận xét

Trong bài toán này, nếu cố duy trì trực tiếp các hệ số a, b của $\text{cost}(i, k, j)$ thì bài toán sẽ phức tạp hơn. Vì vậy ta mạnh dạn đổi biến: điều chỉnh đại lượng khác để giữ a, b bất biến, rồi dùng hàng đợi và độ dốc để tối ưu thuật toán xuống $O(NM)$.

4 Tách biến và duy trì bằng sự kiện

4.1 Ví dụ 3: Circular Railway

Trên một đường ray hình tròn dài L có N điểm loại A và N điểm loại B . Cần chọn một cách ghép cặp một-một giữa các điểm A và các điểm B sao cho tổng khoảng cách ngắn nhất trên đường ray giữa các cặp được ghép là nhỏ nhất.

Dữ liệu thỏa

$$1 \leq n \leq 50000, \quad 2 \leq L \leq 10^9.$$

Phân tích

Sắp xếp các điểm loại A theo chiều kim đồng hồ và gọi tọa độ của chúng là $A_1, A_2, \dots, A_n$. Sắp xếp các điểm loại B theo chiều kim đồng hồ và gọi tọa độ của chúng là $B_1, B_2, \dots, B_n$; khi cần trải vòng tròn thành đường thẳng, đặt $B_{i+n} = B_i + L$.

Về bản chất, đây là bài toán ghép cặp trọng số nhỏ nhất trên đồ thị hai phía. Nhưng thuật toán tổng quát có thể tốn $O(n^4)$ trong trường hợp xấu, không phù hợp với giới hạn dữ liệu.

Quan sát kỹ hơn, trong ghép cặp tối ưu có một tính chất quan trọng.

Tính chất 3.1. Hai cạnh ghép cặp không thể cắt nhau.

Chứng minh. Giả sử có hai cạnh ghép $A_i - B_v$ và $A_j - B_u$ cắt nhau. Nếu đổi ghép thành $A_i - B_u$ và $A_j - B_v$, tổng trọng số không tăng, thực tế trong cấu hình cắt nhau sẽ tốt hơn. Điều này mâu thuẫn với tính tối ưu của phương án ban đầu.

Từ tính chất trên, ghép cặp tối ưu phải có dạng sau: nếu A_1 được ghép với B_j , thì với $1 < i \leq n$, điểm A_i sẽ được ghép với điểm loại B thứ i kể từ B_j theo chiều kim đồng hồ, tức B_{i+j-1} .

Do đó ta có thuật toán đầu tiên: liệt kê điểm B_j ghép với A_1 , suy ra toàn bộ các cặp theo thứ tự, rồi tính tổng chi phí của phương án ấy. Giá trị nhỏ nhất trong các phương án là đáp án. Thuật toán này tốn $O(n^2)$. Với $n = 50000$ vẫn còn quá lớn, nhưng đã là một bước tiến đáng kể.

Tiếp tục phân tích. Giả sử A_i được ghép với B_{Q_i} , và đặt C_i là khoảng cách ngắn nhất từ A_i đến B_{Q_i} . Trong thuật toán vừa nêu, nếu duy trì được các C_i thì có thể tính đáp án. Nhưng chỉ riêng việc tính mọi khoảng cách từ A_i đến B_j đã là $O(n^2)$; tức C_i có tới n^2 giá trị khác nhau có thể xuất hiện. Muốn tối ưu, ta phải đi đường khác.

Xét một cặp A_i và B_j . Khoảng cách ngắn nhất C_i có bốn khả năng:

1. $C_i = A_i - B_j$,
2. $C_i = B_j - A_i$,
3. $C_i = L + A_i - B_j$,
4. $C_i = L + B_j - A_i$.

Các điều kiện tương ứng là:

1. $A_i > B_j$ và $A_i - B_j \leq L/2$.
2. $B_j > A_i$ và $B_j - A_i \leq L/2$.
3. $B_j - A_i > L/2$.
4. $A_i - B_j > L/2$.

Ta nhận thấy C_i chỉ phụ thuộc vào quan hệ tương đối giữa A_i và B_j . Nếu không thể trực tiếp duy trì biến C_i , ta tách nó thành hai phần:

$$C_i = f(A_i) + g(B_j).$$

Xét ảnh hưởng của A_i lên C_i , tức hàm $f(A_i)$. Từ bốn trường hợp trên:

$$\begin{aligned} B_j - A_i > L/2 : & \quad f(A_i) = A_i, \\ L/2 \geq B_j - A_i > 0 : & \quad f(A_i) = -A_i, \\ L/2 \geq A_i - B_j > 0 : & \quad f(A_i) = A_i, \\ L/2 < A_i - B_j : & \quad f(A_i) = L - A_i. \end{aligned}$$

Tập trung vào biến $f(A_i)$, ta thấy tuy nó vẫn là một biến, nhưng so với C_i , khi j tăng từ 1 đến N , $f(A_i)$ chỉ có nhiều nhất bốn dạng; thực ra hai trường hợp đầu và cuối không đồng thời xuất hiện theo một vòng duyệt, nên chỉ còn ba lần thay đổi đáng kể. Có thể xem $f(A_i)$ là một đại lượng gần như không đổi. Phân tích tương tự cho $g(B_j)$ cũng cho thấy nó chỉ thay đổi một số lần hằng số.

Bây giờ liệt kê k từ nhỏ đến lớn, trong đó A_1 ghép với B_k , còn các điểm khác ghép lần lượt theo thứ tự. Tổng chi phí hiện tại là

$$Sum = \sum_i C_i = \sum_i (f(A_i) + g(B_i)) = \sum_i f(A_i) + \sum_i g(B_i),$$

trong đó chỉ số của các B được hiểu theo phép dịch vòng tương ứng với k .

Khi k thay đổi, ta chỉ quan tâm đến sự thay đổi của Sum . Theo phân tích trên, trong toàn bộ quá trình liệt kê $k = 1, 2, \dots, n$, mỗi biến $f(A_i)$ và $g(B_i)$ chỉ thay đổi nhiều nhất bốn lần. Ta có thể tiền xử lý thời điểm xảy ra các thay đổi ấy. Chẳng hạn với mỗi A_i , biết được thời điểm $k = x_1$ đổi từ trường hợp 1 sang 2, thời điểm $k = x_2$ đổi từ 2 sang 3, và thời điểm $k = x_3$ đổi từ 3 sang 4. Với $g(B_i)$ cũng xử lý tương tự.

Mỗi lần $f(A_i)$ hoặc $g(B_i)$ thay đổi được xem là một sự kiện. Tổng số sự kiện không vượt quá $8n$. Nếu đã biết thời điểm của từng sự kiện, trong đó thời điểm x nghĩa là sự kiện xảy ra khi k chuyển từ x sang $x + 1$, ta có thể duy trì Sum tức thời. Giá trị nhỏ nhất của Sum trong quá trình duyệt chính là đáp án.

Quy trình thuật toán:

1. Sắp xếp các tọa độ A_i và B_i .
2. Tiền xử lý thời điểm xảy ra của mọi sự kiện.
3. Duyệt k từ 1 đến N ; dùng bảng băm hoặc các danh sách theo thời điểm để lấy các sự kiện xảy ra tại thời điểm hiện tại, đồng thời cập nhật Sum .
4. In giá trị nhỏ nhất của Sum .

Việc tiền xử lý các thời điểm x_1, x_2, x_3 thực chất là tìm, với mỗi A_i , khoảng chỉ số j của B_j ứng với bốn trường hợp trên. Do các dãy A_i và B_i đều đã được sắp xếp, có thể dùng con trỏ trượt để duy trì các khoảng này.

Độ phức tạp thời gian là

$$O(\text{sắp xếp} + \text{tiền xử lý} + \text{số sự kiện}) = O(n \log n + n + 8n) = O(n \log n).$$

Nhìn lại toàn bộ quá trình, ta thấy chỉ cần nắm chắc bản chất bài toán và giữ lấy biến quan trọng nhất là tổng chi phí, rồi tách mỗi hạng C_i thành hai phần không phụ thuộc lẫn nhau, ta có thể lần lượt thảo luận hai “hằng lượng” $f(A_i)$ và $g(B_i)$, cuối cùng tối ưu thuật toán đến độ phức tạp thỏa đáng.

5 Tổng kết

Bài viết đã nêu ba ví dụ, giới thiệu kỹ thuật biến các biến lượng thành phần “bất biến” trong tin học. Thực tế kỹ thuật này có tác dụng ở rất nhiều nơi. Do khuôn khổ có hạn, bài viết chỉ trình bày ba phương pháp tương đối điển hình. Từ các bài toán này cũng có thể thấy, việc dùng kỹ thuật ấy không chỉ là rút gọn đơn giản; quan hệ giữa một số biến ẩn rất sâu. Cần quan sát cẩn thận, mạnh dạn phỏng đoán và phân tích sâu thì mới tìm được “hằng lượng” phù hợp.

Tài liệu tham khảo

1. ZJU Online Judge 2376, *Ants*. [ZJU problem 2376](#).
2. PKU Online Judge 2841, *Navigation Game*. [PKU problem 2841](#).
3. Tang Ze. *Sơ lược về ứng dụng của hàng đợi trong một lớp bài toán có tính đơn điệu*. Luận văn đội tuyển tập huấn, 2006.
4. SGU Online Judge 313, *Circular Railway*. [SGU problem 313](#).
5. Liu Rujia, Huang Liang. *Nghệ thuật thuật toán và thi tin học*. Nhà xuất bản Đại học Thanh Hoa, 2003.